Qualidade de Software e Programação Orientada a Objetos

Material Didático: Java, Eclipse, Maven e JUnit 5

Material de apoio didático – Engenharia de Software

Outubro de 2026

Fonte principal: *LIVRO_UNICO-EDS*, Unidade 3, "Gerenciamento de qualidade de software", páginas impressas 111–146. Os exemplos de Sistema de Biblioteca em Java são aplicações didáticas dos conceitos apresentados no livro.

Sumário

1 A gestão da qualidade no processo de desenvolvimento de software 2

1.1 Conceito de qualidade de software . . . 2
1.2 Características importantes . . . 2
1.3 Sistema de gestão da qualidade . . . 2
1.4 Principais processos da gestão da qualidade . . . 3

2 A garantia da qualidade do software 3

2.1 Conceito de SQA . . . 3
2.2 Atividades principais . . . 3
2.3 Características de qualidade do produto na ISO/IEC 25010:2011 . . . 4
2.4 Aspectos que merecem atenção . . . 4
2.5 Caso citado no material: Ariane 5 . . . 5

3 As normas de qualidade aplicadas no desenvolvimento de software 5

3.1 Por que utilizar normas e modelos? . . . 5
3.2 Normas e modelos apresentados no livro . . . 5
3.2.1 ISO 9001:2015 – Sistema de Gestão da Qualidade . . . 5
3.2.2 ISO/IEC 90003:2014 – Orientações para software . . . 5
3.2.3 ISO/IEC 25010:2011 – Qualidade do produto . . . 6
3.2.4 CMMI – Melhoria da maturidade de processos . . . 6
3.2.5 MPS.BR – Melhoria de Processo do Software Brasileiro . . . 6
3.3 Critérios para escolher um modelo de qualidade . . . 6

4 Comparação entre as três seções **7**

5 Aplicação integradora: Sistema de Biblioteca em Java **7**

6 Questões para discussão **7**

Considerações finais **8**

7 Implementação em Java: roteiro prático no Eclipse **9**
7.1 Objetivos e ligação com as seções teóricas 9
7.2 Pré-requisitos 9
7.3 Estrutura do repositório 9
7.4 Código 01: Livro.java – encapsulamento e adequação funcional 9
7.5 Código 02: Usuario.java – organização de responsabilidades 10
7.6 Código 03: Emprestimo.java – validação e exceções 11
7.7 Código 04: Main.java – execução da aplicação 11

8 Testes automatizados e garantia da qualidade **12**
8.1 Código 05: LivroTest.java – testes com JUnit 5 12
8.2 Código 06: EmprestimoTest.java – teste de saídas e validação 13
8.3 Configuração do Maven 13
8.4 Execução pelo Eclipse ou terminal 14

9 Normas e modelos de qualidade aplicados ao projeto **15**
9.1 Avaliação orientada pela ISO/IEC 25010:2011 15
9.2 Gestão de processos: ISO 9001, ISO/IEC 90003, CMMI e MPS.BR 15

10 Atividade avaliativa e documentação do GitHub **15**
10.1 Entrega solicitada aos estudantes 15
10.2 Critérios sugeridos de avaliação 16
10.3 Perguntas para discussão 16
10.4 Arquivos complementares no repositório 16

Referência didática e delimitação **16**

1 A gestão da qualidade no processo de desenvolvimento de software

Referência: Seção 3.1, páginas 111–122 do livro.

1.1 Conceito de qualidade de software

Qualidade de software é a capacidade de um sistema atender aos requisitos definidos e às necessidades dos usuários, funcionando de acordo com sua finalidade. O material ressalta que qualidade não é sinônimo de perfeição, não é um conceito absoluto e deve ser avaliada por critérios suficientemente objetivos.

- **Conformidade com requisitos:** o software realiza as funções especificadas.
- **Adequação ao propósito:** o produto atende ao problema para o qual foi criado.
- **Satisfação do usuário:** a experiência de utilização também influencia a percepção de qualidade.
- **Critérios de medição:** estabelecer níveis aceitáveis e evidências verificáveis.

Exemplo prático. No Sistema de Biblioteca, registrar corretamente empréstimos, impedir operações inválidas e localizar obras com facilidade são requisitos ligados à qualidade.

1.2 Características importantes

Característica	Conceito	Exemplo
Corretude	Executar conforme o especificado.	Calcular corretamente uma multa por atraso.
Eficiência	Usar adequadamente recursos computacionais.	Consultar o catálogo sem processamento excessivo.
Usabilidade	Ser fácil de compreender e utilizar.	Tela clara para cadastrar leitores.
Portabilidade	Facilitar a execução em outras plataformas.	Executar o programa em Windows e Linux.
Interoperabilidade	Permitir a troca de informações com outros sistemas.	Integrar-se ao cadastro institucional de alunos.

1.3 Sistema de gestão da qualidade

O livro apresenta a gestão da qualidade como um sistema integrado à organização, envolvendo pessoas, processos e ferramentas. Conforme o modelo citado de Moorthy, pode ser estruturado em quatro níveis:

1. **Manual de qualidade:** políticas e diretrizes da organização.
2. **Métodos e processos:** procedimentos utilizados para desenvolver e entregar o produto.

3. **Linhas principais, checklists e modelos:** instrumentos padronizados para o trabalho cotidiano.
4. **Registros e documentos:** evidências das atividades, verificações e validações realizadas.

Exemplo prático. Uma equipe Java pode definir convenções de codificação, manter checklist de revisão, documentar os testes e registrar os resultados de cada entrega.

1.4 Principais processos da gestão da qualidade

- **Planejamento da qualidade:** definir características desejadas do produto e organizar os processos necessários para obtê-las.
- **Garantia da qualidade (SQA):** verificar a conformidade de processos e produtos com os requisitos e planos.
- **Verificação e Validação (V&V):** avaliar artefatos durante o ciclo de vida e verificar a adequação do produto às necessidades.
- **Revisões técnicas:** examinar artefatos em relação às especificações e aos padrões.
- **Inspeções:** identificar anomalias de forma estruturada, com condução imparcial.

Exemplo prático. Antes de liberar o módulo de empréstimos, a equipe verifica regras de negócio, revisa o código, executa testes e registra eventuais defeitos.

Síntese: a qualidade precisa ser incorporada durante a construção do software, não apenas examinada no final.

2 A garantia da qualidade do software

Referência: Seção 3.2, páginas 123–134 do livro.

2.1 Conceito de SQA

Software Quality Assurance (SQA), ou Garantia da Qualidade do Software, corresponde ao conjunto planejado e sistemático de ações voltadas a assegurar a qualidade do produto e do processo de desenvolvimento. Participam dessas ações os desenvolvedores, gestores, profissionais de qualidade e o cliente.

2.2 Atividades principais

1. **Aplicação de métodos técnicos:** cuidar da qualidade desde a especificação e o projeto, avaliando requisitos, protótipos e versões entregues.
2. **Revisões técnicas formais:** reunir uma equipe para buscar problemas em partes definidas do produto.

3. **Atividades de testes:** executar casos selecionados para detectar defeitos e avaliar o comportamento do programa.

O processo de teste apresentado no livro abrange quatro etapas:

1. Planejamento;
2. Projeto dos casos de teste;
3. Execução;
4. Avaliação dos resultados.

Exemplo prático. Para a devolução de livros, criar um teste de devolução no prazo, outro com atraso e outro para tentativa de devolução de uma obra que não consta como emprestada.

2.3 Características de qualidade do produto na ISO/IEC 25010:2011

O livro distingue características do **produto** e da **qualidade em uso**. A tabela seguinte resume as oito características de qualidade do produto apresentadas no material.

Característica	**Exemplo no Sistema de Biblioteca**
Adequação funcional	Registrar empréstimos e devoluções segundo os requisitos.
Confiabilidade	Manter comportamento consistente e tratar falhas.
Usabilidade	Oferecer navegação simples e mensagens compreensíveis.
Eficiência de desempenho	Responder a consultas em tempo adequado, sem consumo excessivo de recursos.
Segurança	Restringir ações administrativas a usuários autorizados.
Compatibilidade	Trocar dados com sistemas acadêmicos.
Capacidade de manutenção	Alterar regras de multas sem comprometer módulos independentes.
Portabilidade	Facilitar instalação e execução em ambientes diferentes.

As características de **qualidade em uso** listadas no material são: efetividade, eficiência, satisfação, uso sem riscos e cobertura de contexto.

2.4 Aspectos que merecem atenção

Confiabilidade.
O programa deve se comportar de forma consistente, inclusive em situações incomuns. **Exemplo:** uma queda de conexão não pode deixar um empréstimo parcialmente registrado.

Segurança.
Dados e funcionalidades devem ser protegidos de acessos não autorizados. **Exemplo:** só funcionários autorizados podem excluir registros.

Usabilidade.
O sistema deve facilitar a operação e ajudar o usuário a evitar erros. **Exemplo:** avisar claramente que uma obra está indisponível.

Capacidade de manutenção.
O software deve favorecer análise, alterações e testes. **Exemplo:** separar as classes `Livro`, `Usuario` e `Emprestimo` com responsabilidades claras.

2.5 Caso citado no material: Ariane 5

O livro relata a falha do foguete Ariane 5, em 1996, associada a um erro de conversão numérica de ponto flutuante de 64 bits para inteiro de 16 bits, sem proteção adequada contra um valor fora do intervalo permitido. O caso ilustra a necessidade de validar limites, tratar exceções e testar condições anormais. **Síntese:** garantir qualidade exige prevenção, revisões e testes ao longo de todo o ciclo de desenvolvimento.

3 As normas de qualidade aplicadas no desenvolvimento de software

Referência: Seção 3.3, páginas 135–146 do livro.

3.1 Por que utilizar normas e modelos?

Normas e modelos de qualidade oferecem referências mais objetivas para a avaliação de produtos e processos. Ajudam a padronizar atividades, organizar responsabilidades e direcionar melhorias.

3.2 Normas e modelos apresentados no livro

3.2.1 ISO 9001:2015 – Sistema de Gestão da Qualidade

Define requisitos de gestão da qualidade e enfatiza a satisfação do cliente e a melhoria contínua. O livro apresenta sete princípios: foco no cliente, liderança, engajamento de pessoas, abordagem de processo, melhoria, tomada de decisão baseada em evidências e gestão de relacionamento.

Exemplo prático. Estabelecer um procedimento documentado para levantar requisitos, aprovar entregas e tratar reclamações dos usuários.

3.2.2 ISO/IEC 90003:2014 – Orientações para software

Fornece orientações para aplicar a ISO 9001 às atividades de aquisição, fornecimento, desenvolvimento, operação e manutenção de software e serviços relacionados.

Exemplo prático. Definir critérios de revisão, teste, entrega e manutenção adequados a projetos Java.

3.2.3 ISO/IEC 25010:2011 – Qualidade do produto

Apresenta características para avaliar a qualidade de software, incluindo adequação funcional, segurança, usabilidade, confiabilidade e capacidade de manutenção.

Exemplo prático. Avaliar o Sistema de Biblioteca quanto à confiabilidade das operações, segurança de acesso e clareza de interface.

3.2.4 CMMI – Melhoria da maturidade de processos

O CMMI visa melhorar a capacidade e a maturidade dos processos organizacionais. A versão 1.3 abordada no livro apresenta representações *em estágios* e *contínua*.

Nível	Maturidade	Descrição sintética
1	Inicial	Processos pouco definidos; dependência de esforços individuais.
2	Gerenciado	Processos planejados e acompanhados.
3	Definido	Processos padronizados e documentados.
4	Quantitativamente gerenciado	Processos controlados com medidas quantitativas.
5	Em otimização	Melhoria contínua baseada em medições.

Exemplo prático. Uma equipe passa de um desenvolvimento informal para procedimentos de estimativa, documentação, revisão e acompanhamento sistemático.

3.2.5 MPS.BR – Melhoria de Processo do Software Brasileiro

O MPS.BR, apresentado no material como modelo concebido para o contexto nacional, busca promover a melhoria de processos em empresas brasileiras. O livro descreve sete níveis de maturidade:

Nível	Denominação
G	Parcialmente gerenciado
F	Gerenciado
E	Parcialmente definido
D	Largamente definido
C	Definido
B	Gerenciado quantitativamente
A	Em otimização

Exemplo prático. Uma pequena empresa pode iniciar pela organização do planejamento e controle de seus projetos e avançar gradualmente em maturidade.

3.3 Critérios para escolher um modelo de qualidade

O livro sugere avaliar:

- custo financeiro da implantação e da avaliação;

- reconhecimento do modelo no mercado;
- adequação à dimensão e à cultura organizacional;
- necessidade de treinamento e apoio especializado;
- objetivos, prazo e benefícios esperados.

Observação temporal: as versões indicadas neste material correspondem àquelas utilizadas pelo livro e não representam, necessariamente, as edições vigentes atualmente.

4 Comparação entre as três seções

Tema	Pergunta orientadora	Resultado esperado
Gestão da qualidade	Como organizar a qualidade no desenvolvimento?	Processos, responsabilidades e critérios definidos.
Garantia da qualidade	Como assegurar e verificar a qualidade?	Revisões, testes e identificação de defeitos.
Normas de qualidade	Que referências usar na padronização?	Adoção de normas e modelos adequados à empresa.

5 Aplicação integradora: Sistema de Biblioteca em Java

Dimensão	Ação prática	Evidência
Gestão da qualidade	Definir requisitos, responsáveis e padrões de código.	Documento de requisitos e checklist.
Garantia da qualidade	Revisar classes Java e testar cadastro, empréstimo e devolução.	Relatório de testes e defeitos.
Normas e modelos	Avaliar características da ISO/IEC 25010 e planejar melhoria de processos.	Relatório de avaliação e plano de melhoria.

6 Questões para discussão

1. Um software que executa corretamente suas funções, mas apresenta uma interface difícil de utilizar, pode ser considerado de qualidade? Justifique com base nas características de qualidade do produto.
2. Qual é a diferença entre gestão da qualidade e garantia da qualidade? Apresente duas atividades práticas de cada uma no desenvolvimento de um Sistema de Biblioteca.
3. Por que uma empresa de software deveria adotar um modelo como CMMI ou MPS.BR? Relacione sua resposta aos processos e à satisfação do cliente.

Considerações finais

A qualidade do software depende da adequação aos requisitos, da organização dos processos, das verificações efetuadas durante o desenvolvimento e dos parâmetros utilizados para avaliar o produto e orientar melhorias. A gestão estrutura o trabalho; a garantia da qualidade acompanha e avalia as práticas; e normas e modelos fornecem referências para a evolução de processos e produtos.

7 Implementação em Java: roteiro prático no Eclipse

7.1 Objetivos e ligação com as seções teóricas

Esta parte apresenta exemplos **produzidos para fins didáticos**, inspirados nos conceitos das Seções 3.1, 3.2 e 3.3 do livro, mas **não transcritos da obra**. A Seção 3.1 orienta a organização e padronização do código; a 3.2 fundamenta a revisão e os testes; a 3.3 oferece referências para avaliar o produto e melhorar o processo. Os mecanismos da linguagem Java ajudam a concretizar esses objetivos, mas não equivalem, por si só, à adoção formal de uma norma de qualidade.

7.2 Pré-requisitos

Utilize um JDK 17 ou superior, Eclipse IDE com suporte a Java, Apache Maven e acesso às dependências do JUnit 5. Na IDE, selecione `File > Import > Maven > Existing Maven Projects`, indique a raiz do repositório e conclua a importação. Os menus podem variar ligeiramente conforme a versão do Eclipse.

7.3 Estrutura do repositório

```
biblioteca-qualidade-java/
|-- src/
|   |-- main/java/br/edu/biblioteca/
|   |   |-- Livro.java
|   |   |-- Usuario.java
|   |   |-- Emprestimo.java
|   |   `-- Main.java
|   `-- test/java/br/edu/biblioteca/
|       |-- LivroTest.java
|       `-- EmprestimoTest.java
|-- docs/
|   |-- 01-gestao-qualidade.md
|   |-- 02-garantia-qualidade.md
|   |-- 03-normas-qualidade.md
|   `-- relatorio-testes.md
|-- latex/material-didatico.tex
|-- pom.xml
`-- README.md
```

7.4 Código 01: Livro.java – encapsulamento e adequação funcional

O estado da disponibilidade só pode ser modificado por métodos da classe. Isso protege a regra de que não é possível emprestar duas vezes um mesmo exemplar sem devolução intermediária.

```
1 package br.edu.biblioteca;
2
3 public class Livro {
4     private final String titulo;
```

```
    private boolean disponivel;

    public Livro(String titulo) {
        if (titulo == null || titulo.isBlank()) {
            throw new IllegalArgumentException("Titulo obrigatorio.");
        }
        this.titulo = titulo;
        this.disponivel = true;
    }

    public String getTitulo() { return titulo; }
    public boolean isDisponivel() { return disponivel; }

    public boolean emprestar() {
        if (!disponivel) return false;
        disponivel = false;
        return true;
    }

    public boolean devolver() {
        if (disponivel) return false;
        disponivel = true;
        return true;
    }
}
```

Listing 1: Livro.java: encapsulamento e transições de estado.

Relação com a qualidade: adequação funcional, manutenibilidade e testabilidade. O encapsulamento é um recurso de projeto, não substitui autenticação nem proteção de dados.

7.5 Código 02: Usuario.java – organização de responsabilidades

```
package br.edu.biblioteca;

public class Usuario {
    private final String nome;
    private final String matricula;

    public Usuario(String nome, String matricula) {
        if (nome == null || nome.isBlank()
                || matricula == null || matricula.isBlank()) {
            throw new IllegalArgumentException("Dados obrigatorios.");
        }
        this.nome = nome;
        this.matricula = matricula;
    }

    public String getNome() { return nome; }
    public String getMatricula() { return matricula; }
}
```

Listing 2: Usuario.java: classe de domínio independente.

Relação com a qualidade: separação de responsabilidades, modularidade e validação

de entradas. A existência de uma classe para cada entidade não garante, isoladamente, baixo acoplamento em sistemas maiores.

7.6 Código 03: Emprestimo.java – validação e exceções

O exemplo conserva o comportamento introdutório da atividade: o método valida o prazo e imprime mensagens. Ele ainda **não registra um empréstimo persistente** nem vincula um livro a um usuário.

```java
package br.edu.biblioteca;

public class Emprestimo {
    public void registrar(int dias) {
        try {
            if (dias <= 0) {
                throw new IllegalArgumentException(
                    "Prazo deve ser positivo.");
            }
            System.out.println("Emprestimo registrado.");
        } catch (IllegalArgumentException e) {
            System.out.println("Erro: " + e.getMessage());
        }
    }
}
```

Listing 3: Emprestimo.java: proteção contra prazo inválido.

Nota de projeto: em sistemas reais, convém separar a regra de negócio da saída para o console e permitir que a camada de apresentação trate a exceção. Aqui a construção `try-catch` foi mantida intencionalmente para demonstrar o mecanismo da linguagem.

7.7 Código 04: Main.java – execução da aplicação

```java
package br.edu.biblioteca;

public class Main {
    public static void main(String[] args) {
        Livro livro = new Livro("Engenharia de Software");
        Usuario usuario = new Usuario("Ana", "2026001");

        System.out.println("Usuario: " + usuario.getNome());
        System.out.println("Livro: " + livro.getTitulo());
        System.out.println("Disponivel: " + livro.isDisponivel());
        System.out.println("Emprestar: " + livro.emprestar());
        System.out.println("Disponivel: " + livro.isDisponivel());

        Emprestimo emprestimo = new Emprestimo();
        emprestimo.registrar(7);
        emprestimo.registrar(-2);
    }
}
```

Listing 4: Main.java: demonstração de objetos e operações.

Saída esperada (representação textual; não é um registro de execução efetiva):

```
Usuario: Ana
Livro: Engenharia de Software
Disponivel: true
Emprestar: true
Disponivel: false
Emprestimo registrado.
Erro: Prazo deve ser positivo.
```

8 Testes automatizados e garantia da qualidade

8.1 Código 05: LivroTest.java – testes com JUnit 5

Os testes avaliam diferentes estados do objeto, aplicando asserções a comportamentos esperados.

```java
package br.edu.biblioteca;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class LivroTest {
    @Test
    void deveEmprestarLivroDisponivel() {
        Livro livro = new Livro("Java POO");
        assertTrue(livro.emprestar());
        assertFalse(livro.isDisponivel());
    }

    @Test
    void naoDeveEmprestarDuasVezes() {
        Livro livro = new Livro("Engenharia de Software");
        assertTrue(livro.emprestar());
        assertFalse(livro.emprestar());
    }

    @Test
    void deveDevolverLivroEmprestado() {
        Livro livro = new Livro("Qualidade de Software");
        assertTrue(livro.emprestar());
        assertTrue(livro.devolver());
        assertTrue(livro.isDisponivel());
    }

    @Test
    void deveRejeitarTituloVazio() {
        assertThrows(IllegalArgumentException.class,
            () -> new Livro("  "));
    }
}
```

Listing 5: LivroTest.java: testes para disponibilidade, empréstimo e devolução.

8.2 Código 06: EmprestimoTest.java – teste de saídas e validação

A implementação didática imprime mensagens no console. Para automatizar a verificação, os testes capturam temporariamente a saída padrão e restauram-na em seguida.

```java
package br.edu.biblioteca;

import static org.junit.jupiter.api.Assertions.assertTrue;
import java.io.ByteArrayOutputStream;
import java.io.PrintStream;
import java.nio.charset.StandardCharsets;
import org.junit.jupiter.api.Test;

public class EmprestimoTest {
    private String capturarSaida(int dias) {
        PrintStream original = System.out;
        ByteArrayOutputStream dados = new ByteArrayOutputStream();
        try (PrintStream captura = new PrintStream(dados, true,
                StandardCharsets.UTF_8)) {
            System.setOut(captura);
            new Emprestimo().registrar(dias);
        } finally {
            System.setOut(original);
        }
        return dados.toString(StandardCharsets.UTF_8);
    }

    @Test
    void deveAceitarPrazoPositivo() {
        assertTrue(capturarSaida(7).contains(
                "Emprestimo registrado."));
    }

    @Test
    void deveInformarPrazoInvalido() {
        assertTrue(capturarSaida(-2).contains(
                "Prazo deve ser positivo."));
    }
}
```

Listing 6: EmprestimoTest.java: verificação das mensagens emitidas.

Limitação: o redirecionamento de `System.out` não é adequado a testes paralelos. No projeto inicial, os testes devem ser executados sequencialmente; uma evolução recomendada é retornar um resultado estruturado, sem acoplar a regra de negócio à interface de console.

8.3 Configuração do Maven

Crie o arquivo `pom.xml` na raiz do projeto. As versões abaixo são escolhas fixas para permitir reprodução do exemplo, não uma afirmação de que sejam as mais recentes.

Listing 7: pom.xml: dependências de testes e compilação.

```xml
<project xmlns="http://maven.apache.org/POM/4.0.0"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
 https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
 <modelVersion>4.0.0</modelVersion>
 <groupId>br.edu.biblioteca</groupId>
 <artifactId>biblioteca-qualidade-java</artifactId>
 <version>1.0-SNAPSHOT</version>
 <properties>
  <maven.compiler.release>17</maven.compiler.release>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
 </properties>
 <dependencies>
  <dependency>
   <groupId>org.junit.jupiter</groupId>
   <artifactId>junit-jupiter</artifactId>
   <version>5.11.4</version>
   <scope>test</scope>
  </dependency>
 </dependencies>
 <build><plugins>
  <plugin>
   <groupId>org.apache.maven.plugins</groupId>
   <artifactId>maven-compiler-plugin</artifactId>
   <version>3.13.0</version>
  </plugin>
  <plugin>
   <groupId>org.apache.maven.plugins</groupId>
   <artifactId>maven-surefire-plugin</artifactId>
   <version>3.5.2</version>
  </plugin>
 </plugins></build>
</project>
```

8.4 Execução pelo Eclipse ou terminal

No Eclipse, clique com o botão direito em `Main.java` e selecione `Run As > Java Application`. Para os testes, selecione a classe de teste e use `Run As > JUnit Test`. Outra opção é executar no terminal, a partir da raiz do repositório:

```
mvn clean test
```

Depois, consulte a visualização do JUnit ou os relatórios que o Maven produzir em `target/surefire-reports/`. O quadro abaixo descreve *casos previstos*, não resultados verificados.

Caso	Situação testada	Resultado esperado
CT-01	Livro recém-criado	Disponível
CT-02	Primeiro empréstimo	Operação aceita
CT-03	Segundo empréstimo sem devolução	Operação rejeitada
CT-04	Devolução após empréstimo	Operação aceita
CT-05	Título vazio ou em branco	Exceção
CT-06	Prazo positivo	Mensagem de registro
CT-07	Prazo negativo	Mensagem de erro

9 Normas e modelos de qualidade aplicados ao projeto

9.1 Avaliação orientada pela ISO/IEC 25010:2011

O livro descreve um conjunto de características de qualidade de produto e de qualidade em uso. A tabela apresenta vínculos *didáticos* entre propriedades do projeto e características da versão abordada no livro, sem presumir conformidade com a norma.

Característica	**Evidência possível**	**Limitação da versão inicial**
Adequação funcional	Testes de empréstimo/devolução	Não há persistência nem cadastro real
Confiabilidade	Validação de prazo	Não há recuperação de falhas externas
Capacidade de manutenção	Classes com papéis separados	Projeto pequeno, sem medição de acoplamento
Usabilidade	Mensagens de console	Não há interface de usuário completa
Segurança	Nenhuma evidência suficiente	Não há autenticação ou autorização
Eficiência de desempenho	Não avaliada	Sem ensaios ou métricas de tempo

9.2 Gestão de processos: ISO 9001, ISO/IEC 90003, CMMI e MPS.BR

A ISO 9001 trata da gestão da qualidade em âmbito organizacional; a ISO/IEC 90003 apresenta orientações para aplicar princípios de gestão em contextos de software. CMMI e MPS.BR orientam práticas de avaliação e melhoria de processos. Neste projeto didático, checklists, revisões por colegas, histórico de alterações e registro de testes podem representar práticas coerentes com tais objetivos, sem configurar certificação, avaliação oficial ou adoção integral de qualquer modelo.

10 Atividade avaliativa e documentação do GitHub

10.1 Entrega solicitada aos estudantes

1. Criar ou importar o projeto Maven no Eclipse e executar `Main.java`.
2. Implementar e analisar as classes `Livro`, `Usuario` e `Emprestimo`.
3. Executar os testes do JUnit e registrar o resultado real, incluindo o ambiente, o número de testes e eventuais falhas.
4. Identificar uma melhoria de qualidade e apresentá-la em uma alteração de código revisável.
5. Preencher `docs/relatorio-testes.md` e relacionar ao menos três características de qualidade com evidências concretas.

10.2 Critérios sugeridos de avaliação

Critério	Peso
Implementação correta e organização em POO	30%
Testes automatizados e interpretação dos resultados	30%
Avaliação justificada da qualidade	20%
Documentação e clareza do repositório	20%

10.3 Perguntas para discussão

1. Por que um atributo `private` ajuda a controlar o estado de um objeto, mas não constitui uma solução completa de segurança?
2. O que ocorre quando chamamos `emprestar()` duas vezes para o mesmo objeto `Livro`?
3. Qual é a vantagem de automatizar a verificação com JUnit em relação a apenas inspecionar a saída do método `main`?
4. Como o projeto poderia evoluir para registrar empréstimos associados a livros, usuários e datas?
5. Quais evidências adicionais seriam necessárias para avaliar segurança, desempenho e usabilidade?

10.4 Arquivos complementares no repositório

Para registrar os resultados, utilize os arquivos da pasta `docs/`: `01-gestao-qualidade.md`, `02-garantia-qualidade.md`, `03-normas-qualidade.md` e `relatorio-testes.md`. O relatório de testes é um **modelo em branco**: deve ser preenchido somente após a execução efetiva do projeto.

Referência didática e delimitação

Texto-base: *LIVRO_UNICO-EDS*, Unidade 3, "Gerenciamento de qualidade de software", Seções 3.1 a 3.3, páginas impressas 111–146. Os exemplos Java, o roteiro Maven/Eclipse, a atividade e os critérios de avaliação são complementos elaborados para este repositório. As versões de normas mencionadas acompanham o recorte histórico do livro e não devem ser interpretadas como necessariamente vigentes em 2026.